PUC-Minas - Ciência da Computação
AED1 – Estudo Dirigido 15

Tema: Introdução à programação VI
Atividade: Apontadores

INSTRUÇÕES:

- Desenvolver classes/métodos em C++ para atender às especificações abaixo.
- Providenciar a documentação essencial:
  nome e matrícula,
  identificação, objetivo, parâmetros e condições especiais,
  se houver, e relatório de testes (exemplos de valores usados e condições testadas).

SUGESTÃO: Montar um menu para a escolha do método a ser testado
(ver modelo em Lista00.cpp).
Testes deverão ser realizados e os valores usados deverão
ser guardados no final do programa como comentários (/* e */).
O uso de recursão é opcional; se desejar utilizá-lo,
fazer também a implementação da forma não-recursiva.

0.) Editar programa em C++, na mesma pasta, cujo nome será Exemplo1500.cpp,
para testar definições de métodos a serem desenvolvidos:

```
/*
  Exemplo1500 - v0.0. - __ / __ / _____
  Author: _____

 */

// ------------------------------------------------ preparacao

// dependências

#include <iostream>

// ------------------------------------------------ definicoes globais

using namespace std;

// ------------------------------------------------ metodos

/**
  Method_00 - nao faz nada.
 */
void method_00 ( )
{
 // nao faz nada
} // end method_00 ( )

/**
  Method_01 - Testar definicoes da classe.
 */
void method_01 ( )
{
 // definir dados

 // identificar
   cout << "\nMethod_01 - v0.0\n" << endl;

 // encerrar
   pause ( "Apertar ENTER para continuar" );
} // end method_01 ( )
```

```
// ---------------------------------------------------- acao principal

/*
  Funcao principal.
  @return codigo de encerramento
*/
int main ( int argc, char** argv )
{
 // definir dado
    int x = 0;                  // definir variavel com valor inicial

 // repetir até desejar parar
    do
    {
     // identificar
        cout << "EXEMPLO1500 - Programa - v0.0\n     " << endl;

     // mostrar opcoes
        cout << "Opcoes                                  " << endl;
        cout << " 0 - parar                              " << endl;
        cout << " 1 - testar definicoes                  " << endl;

     // ler do teclado
        cout << endl << "Entrar com uma opcao: ";
        cin  >> x;

     // escolher acao
        switch ( x )
        {
          case 0:
            method_00 ( );
           break;
          case 1:
            method_01 ( );
           break;
          default:
            cout << endl << "ERRO: Valor invalido." << endl;
        } // end switch
    }
    while ( x != 0 );

 // encerrar
    pause  ( "Apertar ENTER para terminar" );
    return ( 0 );
} // end main ( )
```

/*

---------- documentacao complementar

---------- notas / observacoes / comentarios

---------- previsao de testes

---------- historico

Versao	Data	Modificacao
0.1	__/__	esboco

---------- testes

Versao	Teste	
0.1	01. (OK)	identificacao de programa

***/**

Exercícios:

DICAS GERAIS: Consultar o Anexo CPP 02 na apostila para outros exemplos.
Não usar métodos ou funções prontos em bibliotecas nativas da linguagem C.
Prever, realizar e registrar todos os testes efetuados.
- Desenvolver e testar cada um dos protótipos de métodos sugeridos abaixo, usando apenas apontadores.
Integrar as chamadas de todos os testes em um só programa.
Os métodos deverão para C++ e buscar compatibilidade com a linguagem C.
Restrições:
- Usar programação orientada para objetos (OOP).
- Usar definições de dados e métodos em classes, e fazer derivação, se possível, de uma classe para tratar erros.
- Incluir método para testar as funcionalidades e erros de manipulações de dados.
- Acrescentar ao modelo sensores para situações de armazenador vazio (ou cheio).
- Não usar *break* ou ***continue***!

01.)
```
/**
    Funcao para determinar o tamanho de uma pilha (LIFO)
    montada sobre arranjo de capacidade máxima definida por constante da classe.
    @return  quantidade de dados na pilha
 */
    int intStack::length ( )
```

02.)
```
/**
    Procedimento para duplicar valor no topo de uma pilha (LIFO)
    montada sobre arranjo. Usar apenas métodos para empilhar e desempilhar.
 */
    void intStack::dup ( )
```

03.)
```
/**
    Procedimento para trocar a ordem dos valores no topo de uma pilha (LIFO)
    montada sobre arranjo. Usar apenas métodos para empilhar e desempilhar.
  */
    void intStack::swap ( )
```

04.)
```
/**
   Procedimento para inverter a ordem dos valores em uma pilha (LIFO)
    montada sobre arranjo, se houver pelo menos três valores empilhados.
    Usar apenas métodos para empilhar e desempilhar.
    DICA: Usar outra(s) pilha(s) como armazenador(es) temporário(s).
 */
    void intStack::invert ( )
```

05.)

```
/**
   Função para procurar certo valor em uma pilha (LIFO)
    montada sobre arranjo. Usar apenas métodos para empilhar e desempilhar.
    DICA: Usar outra(s) pilha(s) como armazenador(es) temporário(s).
 */
   void intStack::search ( int value )
```

06.)

```
/**
    Procedimento para trocar o fundo da pilha (LIFO) por um valor igual à metade,
    montada sobre arranjo. Usar apenas métodos para empilhar e desempilhar.
    DICA: Usar outra(s) pilha(s) como armazenador(es) temporário(s).
 */
   void intStack::halfBottom ( )
```

07.)

```
/**
    Função para remover valor no meio da pilha (LIFO), se houver mais que dois valores,
    montada sobre arranjo. Usar apenas métodos para empilhar e desempilhar.
    DICA: Usar outra(s) pilha(s) como armazenador(es) temporário(s).
 */
   int intStack::removeMid ( )
```

08.)

```
/**
    Funcao para comparar duas pilhas de inteiros
    montadas sobre arranjos. Usar apenas métodos para empilhar e desempilhar.
    @return zero      , se forem iguais;
                negativo, se o valor da diferenca for menor e estiver na primeira pilha
                positivo , se o valor da diferenca for maior   e estiver na primeira pilha
 */
   int intStack::compare (  intStack other )
```

09.)

```
/**
    Procedimento para juntar uma segunda pilha à primeira,
    montadas sobre arranjos. Usar apenas métodos para empilhar e desempilhar.
 */
   void intStack::join (  intStack other )
```

10.)

```
/**
    Procedimento para intercalar duas pilhas,
    montadas sobre arranjos. Usar apenas métodos para empilhar e desempilhar.
 */
   void intStack::merge (  intStack other )
```

Tarefas extras

E1.)

```
/**
    Função para achar o menor valor de uma pilha de inteiros,
    usando um valor de cada por vez. Usar apenas métodos para empilhar e desempilhar.
    DICA: Usar outra(s) pilha(s) como armazenador(es) temporário(s).
 */
    int intStack::least ( )
```

E1.)

```
/**
    Função para calcular a média dos valores em uma pilha de inteiros.
    Usar apenas métodos para empilhar e desempilhar.
    DICA: Usar outra(s) pilha(s) como armazenador(es) temporário(s).
 */
    double intStack::average ( )
```